=== UNIT 4: ARRAYS, STRING BASICS & SEARCHING-SORTING ALGORITHMS ===

>> 4.1 Array Data Structure and Memory Layout

An array is a homogeneous, contiguous collection of elements of identical data type stored in contiguous physical memory locations.
* One-Dimensional Arrays: Declaration: int arr[N];. Elements accessed via zero-based indices (0 to N-1). Array indexing formula: Address(arr[i]) = Base_Address + i * sizeof(data_type). Array decay: Array name decays into pointer to first element: arr == &arr[0].
* Multidimensional Arrays (2D Matrices): Declaration: int matrix[ROWS][COLS];.
- Row-Major Order (C Standard): Elements stored consecutively row by row in physical RAM. Address formula:
Address(matrix[i][j]) = Base_Address + [i * COLS + j] * sizeof(data_type).
- Column-Major Order (Used in Fortran/MATLAB): Elements stored column by column.

>> 4.2 Searching Algorithms

* Linear Search: Sequentially examines each array element from index 0 to N-1 until target is matched. Time complexity: Best Case O(1), Worst Case O(N), Average Case O(N). Applicable to unsorted arrays.
* Binary Search: Divide-and-conquer search on sorted arrays. Compares target with middle element mid = low + (high - low)/2. If target < arr[mid], search left half; if target > arr[mid], search right half. Time complexity: Best Case O(1), Worst/Average Case O(log2 N). Space complexity: O(1) iterative.

>> 4.3 Sorting Algorithms

* Bubble Sort: Compares adjacent elements and swaps them if out of order. After pass i, ith largest element bubbles up to final position at end of array. Time Complexity: Best Case O(N) (with swap flag optimization), Worst/Average Case O(N^2). Stable sort.
* Selection Sort: Finds minimum element in unsorted subarray and swaps it with first unsorted element. Time Complexity: O(N^2) in all cases (Best, Worst, Average). Unstable sort.

[PROGRAM 1: Bubble Sort with Early-Termination Flag Optimization]

Problem: Implement Bubble Sort in C with a swapped boolean flag that achieves O(N) linear time on already-sorted arrays, tracking comparisons and swap operations.
Source Code:

```
#include <stdio.h>
#include <stdbool.h>
void bubbleSortOptimized(int arr[], int n) {
int comparisons = 0, swaps = 0;
for (int i = 0; i < n - 1; i++) {
bool swapped = false;
for (int j = 0; j < n - i - 1; j++) {
comparisons++;
if (arr[j] > arr[j + 1]) {
int temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
swapped = true;
swaps++;
}
}
if (!swapped) {
printf("Array sorted early at pass %d!\n", i + 1);
break;
}
}
printf("Total Comparisons: %d | Total Swaps: %d\n", comparisons, swaps);
}
int main() {
int data[] = { 64, 34, 25, 12, 22, 11, 90 };
int n = sizeof(data) / sizeof(data[0]);
bubbleSortOptimized(data, n);
printf("Sorted Array: ");
for (int i = 0; i < n; i++) printf("%d ", data[i]);
printf("\n");
return 0;
}
```

Output: Total Comparisons: 21 | Total Swaps: 14 | Sorted Array: 11 12 22 25 34 64 90.

[PROGRAM 2: Binary Search Implementation with Step-by-Step Interval Tracing]

Problem: Implement iterative Binary Search in C, printing low, high, and mid index values at every iteration to illustrate logarithmic halving.

Source Code:

```
#include <stdio.h>
int binarySearch(int arr[], int n, int key) {
int low = 0, high = n - 1, step = 1;
while (low <= high) {
int mid = low + (high - low) / 2;
```

```
printf("Step %d: low=%d, high=%d, mid=%d, arr[mid]=%d\n", step++, low, high, mid,
arr[mid]);
if (arr[mid] == key)
return mid;
if (arr[mid] < key)
low = mid + 1;
else
high = mid - 1;
}
return -1;
}
int main() {
int sorted[] = { 2, 5, 8, 12, 16, 23, 38, 56, 72, 91 };
int n = 10, target = 23;
int idx = binarySearch(sorted, n, target);
if (idx != -1)
printf("Found target %d at index %d!\n", target, idx);
else
printf("Target not found!\n");
return 0;
}
```

Trace Output:

Step 1: low=0, high=9, mid=4, arr[mid]=16 (16 < 23 => search right half)

Step 2: low=5, high=9, mid=7, arr[mid]=56 (56 > 23 => search left half)

Step 3: low=5, high=6, mid=5, arr[mid]=23 (Match found at index 5!)

[PROGRAM 3: Matrix Multiplication with Dimensional Compatibility Verification]

Problem: Write an ANSI C program to compute matrix product C = A * B of an (m x p) matrix and a (p x n) matrix, verifying dimensional compatibility.

Source Code:

```
#include <stdio.h>
#define MAX 10
void multiplyMatrices(int A[MAX][MAX], int B[MAX][MAX], int C[MAX][MAX], int r1, int c1,
int r2, int c2) {
if (c1 != r2) {
printf("Matrix dimensions incompatible for multiplication!\n");
return;
}
for (int i = 0; i < r1; i++) {
for (int j = 0; j < c2; j++) {
C[i][j] = 0;
for (int k = 0; k < c1; k++) {
C[i][j] += A[i][k] * B[k][j];
```

```
}
}
}
}
int main() {
int A[MAX][MAX] = { {1, 2}, {3, 4} };
int B[MAX][MAX] = { {5, 6}, {7, 8} };
int C[MAX][MAX];
multiplyMatrices(A, B, C, 2, 2, 2, 2);
printf("Product Matrix C (2x2):\n");
for (int i = 0; i < 2; i++) {
for (int j = 0; j < 2; j++)
printf("%4d", C[i][j]);
printf("\n");
}
return 0;
}
```

Output:

```
19 22
43 50
```

[PROGRAM 4: Selection Sort with Step-by-Step Min-Element Tracking]

Problem: Implement Selection Sort in C, printing array after every outer iteration to show unsorted subarray reduction.

Source Code:

```
#include <stdio.h>
void selectionSort(int arr[], int n) {
for (int i = 0; i < n - 1; i++) {
int minIdx = i;
for (int j = i + 1; j < n; j++) {
if (arr[j] < arr[minIdx])
minIdx = j;
}
int temp = arr[minIdx];
arr[minIdx] = arr[i];
arr[i] = temp;
printf("Pass %d (min=%d placed at %d): ", i + 1, temp, i);
for (int k = 0; k < n; k++) printf("%d ", arr[k]);
printf("\n");
}
}
int main() {
int data[] = { 29, 10, 14, 37, 13 };
```

```
selectionSort(data, 5);
return 0;
}
```

Output Steps:
Pass 1 (min=10 placed at 0): 10 29 14 37 13
Pass 2 (min=13 placed at 1): 10 13 14 37 29
Pass 3 (min=14 placed at 2): 10 13 14 37 29
Pass 4 (min=29 placed at 3): 10 13 14 29 37

>> 4.4 University Review & Examination Questions

* Q1: Derive physical memory address mapping formula for a 2D array in Row-Major Order vs. Column-Major Order.
Answer: Let a 2D array be declared as A[M][N] with base address B and element size S bytes. (1) Row-Major Order: Elements of Row 0 stored contiguously, followed by Row 1, up to Row M-1. To access element A[i][j], program skips 'i' complete rows of N elements each, plus 'j' elements within row i. Address(A[i][j]) = B + [i * N + j] * S. (2) Column-Major Order: Elements of Column 0 stored contiguously, followed by Column 1, up to Column N-1. To access A[i][j], program skips 'j' complete columns of M elements each, plus 'i' elements within column j. Address(A[i][j]) = B + [j * M + i] * S.
* Q2: Explain why Binary Search cannot be applied to an unsorted array, and derive its worst-case time complexity.
Answer: Binary Search relies strictly on monotonicity (ordered property) of data. It assumes that if target key > arr[mid], target CANNOT exist in left half [low, mid-1], safely discarding N/2 elements in a single comparison. In an unsorted array, this assumption fails because larger elements could reside anywhere. Derivation of Time Complexity: At each step, search space is divided by 2. After step 1: N/2; step 2: N/4; step k: N / 2^k. Search terminates when 1 element remains: N / 2^k = 1 => 2^k = N => k = log2(N). Therefore, worst-case number of comparisons is proportional to log2(N), giving O(log N) time complexity.

>> 4.5 Key Takeaways and Revision Points

* Arrays allocate contiguous physical memory; 2D arrays in C use Row-Major order.
* Linear search is O(N); Binary search achieves O(log N) on pre-sorted arrays.
* Bubble sort is stable with O(N^2) complexity (optimizable to O(N) using a swap flag).
* Matrix multiplication of (m x p) and (p x n) requires O(m * p * n) operations.